

Betriebssysteme 1 | Bsys1

Zusammenfassung

INHALTSVERZEICHNIS

1. Prozessor	2
1.1. Instruktionen	2
1.1.1. Codierung	2
1.1.2. Sequenzen	2
1.1.3. Takt / Zyklus	3
2. Maschinencode	3
2.1. Intel Terminologie	3
2.2. Formate	3
3. Assembler	4
3.1. Spezifikation von Daten	4
3.1.1. Spezifikation von Zahlen	4
3.2. Bits	5
3.3. Offset	5
3.4. Labels	5
3.5. Flat-Form Binaries	6
3.6. Objekt-Dateien	6
3.6.1. Symboltabelle	6
3.7. Register	6
3.7.1. Allzweckregister	7
3.7.2. Spezialregister	7
3.7.3. Übersicht	7
3.8. Instruktionen	8
3.8.1. Datentransfer-Operationen	8
3.8.2. Adressierung	9
3.9. Linker	10
3.9.1. Einsprungspunkt	10
3.9.2. Syscalls	10
3.9.3. Ende des Programms	10
4. C	10
4.1. Toolchain	10
4.2. Präprozessor	11
4.2.1. Tokenization	11
4.2.2. Präprozessor-Direktiven	11
4.2.3. Includes	11
4.2.4. Macros	12
4.3. Compiler	12
4.3.1. clang commands	12
4.3.2. Deklaration und definition	12
4.3.3. Variablen	13
4.3.4. Quirks	13

1. PROZESSOR

Der Prozessor und Speicher sind getrennt und sind durch einen Speicherbus verbunden. Dieser umfasst:

Adressbus: enthält Adresse der Speicherzelle, auf die zugegriffen werden soll

Datenbus: enthält Daten, die aus Speicherzelle gelesen wurde bzw. in diese geschrieben werden soll

Steuersignale: die z.B. angeben, ob gelesen oder geschrieben werden soll

Schreibvorgang	Lesevorgang
1) Prozessor legt Adresse auf Adressbus	1) Prozessor legt Adresse auf Adressbus
2) Prozessor legt Daten auf Datenbus	2) Prozessor aktiviert Speicherbus zum Lesen
3) Prozessor aktiviert Speicherbus zum Schreiben	3) Speicher legt Daten auf Datenbus

- Jeder Prozessor enthält eine kleine Menge an Speicher: die **Register**
- Jeder Prozessor besteht aus vielen kleinen Bausteinen, die jeweils eine **Operation** durchführen können
 - Operationen können Register und/oder den Speicherbus lesen und/oder schreiben
 - Operationen können 0 bis n Operanden (Register, Adressen, Daten) haben
 - Die Anzahl und Art von Operationen ist immer prozessor-spezifisch, aber auf den meisten Prozessoren ähnlich

1.1. INSTRUKTIONEN

Eine **Instruktion** ist die Kombination einer Operation mit Operanden, z.B. «Kopiere Wert aus Register x in Register y». `mov y, x`

1.1.1. Codierung

Operationen und Register werden durchnummeriert. Gemeinsam mit Daten- und/oder Adressoperanden ergibt sich eine Bytesequenz.

Beispiel 1: «Addiere 12345678_h zu Register 1» auf Intel 64 Architektur

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5
81 _h	C1 _h	78 _h	56 _h	34 _h	12 _h
Opcode		12345678 _h			

Die Gesamtheit aller möglichen (binären) Instruktionscodes eines Prozessors (Maschine) ist sein Maschinencode und wird vom Hersteller festgelegt.

1.1.2. Sequenzen

Die Sequenzen der Instruktionen können an mehreren Orten liegen:

- Hart-codiert im Prozessor: Sequenz kann nach Produktion des Prozessors nie geändert werden
- Im Hauptspeicher: Sequenz kann beliebig geändert werden, muss über den Speicherbus abgefragt werden

Der Prozessor hat ein spezielles Register, das die Adresse des nächsten Befehls im Hauptspeicher enthält (Befehlszeiger / **instruction pointer** IP / **program counter** PC).

1.1.3. Takt / Zyklus

Der gesamte Computer ist getaktet: Alle Bausteine (auch im Prozessor, im Speicher etc) erhalten ein Takt-Signal (**Clock**)

Über mehrere Takte hinweg führt der Prozessor folgende Schritte aus (**Zyklus**):

- 1) Prozessor fordert Instruktion ab der Adresse an, die im Befehlszeiger steht
- 2) Prozessor decodiert Operation und Operanden aus Instruktion
- 3) Prozessor wählt mit Operation korrespondierenden Baustein aus
- 4) Aktiver Baustein liest ggf. aus den Registern
- 5) Aktiver Baustein führt ggf. Berechnung aus
- 6) Aktiver Baustein schreibt ggf. in die Register
- 7) Prozessor erhöht Befehlszeiger entsprechend der Länge der Instruktion

Diese Schritte können stark parallelisiert werden (**Pipelining**)

2. MASCHINENCODE

Intel 64 Prozessoren basieren auf einer Little-Endian-Architektur. Es gibt 2 Modi: 32 Bit (Legacy) und 64 Bit, befassen uns hier aber nur mit 64 Bit.

2.1. INTEL TERMINOLOGIE

Begriff	Definition
Byte	8 Bit
Word	2 Byte / 16 Bit
Doubleword	4 Byte / 32 Bit, auch DWord
Quadword	8 Byte / 64 Bit, auch QWord
Double Quadword	16 Byte / 128 Bit, auch DQWord

Intel-Prozessoren verwenden als kleinste Einheiten einzelne Bytes (8 Bit). Ein einzelnes Bit muss immer mit dem gesamten Byte verarbeitet werden. Dargestellt werden die Bytes als 2 hexadezimale Stellen s_0 und s_1 , wobei die höherwertige Ziffer immer links steht ($s_1 s_0$).

Ein Word hat 4 hexadezimale Stellen s_0 bis s_3 , wobei es üblich ist, die Stellen **von rechts nach links** ($s_3 s_2 s_1 s_0$) zu schreiben.

2.2. FORMATE

Die Bytes werden im **Big-Endian** Format **von rechts nach links** und im **Little-Endian** Format **von links nach rechts** geschrieben.

Big-Endian	Little-Endian												
<table border="1"> <tr> <td>1b</td><td>0b</td></tr> <tr> <td>CA_h</td><td>FE_h</td></tr> <tr> <td>i</td><td>$i + 1$</td></tr> </table>	1b	0b	CA _h	FE _h	i	$i + 1$	<table border="1"> <tr> <td>0b</td><td>1b</td></tr> <tr> <td>FE_h</td><td>CA_h</td></tr> <tr> <td>i</td><td>$i + 1$</td></tr> </table>	0b	1b	FE _h	CA _h	i	$i + 1$
1b	0b												
CA _h	FE _h												
i	$i + 1$												
0b	1b												
FE _h	CA _h												
i	$i + 1$												
Zahlen beginnen mit dem «grossen Ende», dem MSByte (Most Significant)	Zahlen beginnen mit dem «kleinen Ende», dem LSByte (Least Significant)												

Big-Endian

3b	2b	1b	0b
87 _h	65 _h	43 _h	21 _h
i	$i + 1$	$i + 2$	$i + 3$

Little-Endian

0b	1b	2b	3b
21 _h	43 _h	65 _h	87 _h
i	$i + 1$	$i + 2$	$i + 3$

Vorteil von Little-Endian ist, dass man ein DWord mit Nullen in den höherwertigen Stellen als Word oder Byte mit derselben Adresse verwenden kann.

Big-Endian

3b	2b	1b	0b
00 _h	00 _h	00 _h	56 _h
<i>i</i>	<i>i</i> + 1	<i>i</i> + 2	<i>i</i> + 3
			Byte
		Word	
DWord			

Little-Endian

0b	1b	2b	3b
56 _h	00 _h	00 _h	00 _h
<i>i</i>	<i>i</i> + 1	<i>i</i> + 2	<i>i</i> + 3
Byte			
Word			
DWord			

3. ASSEMBLER

Der Assembler ist ein Programm, das textuelle Befehle in Maschinencodes übersetzt. Konventionen hängen massgeblich vom Hersteller des Assemblers ab, selbst für denselben Prozessor. Wir verwenden in diesem Modul den Netwide Assembler [NASM](#).

3.1. SPEZIFIKATION VON DATEN

Quellcode**Resultat im Binär-Output**

db 48	das Byte 48d
db 0x35, 0h21, 049h	die drei Byte 35h, 21h, 49h
db 'a'	das Byte mit dem ASCII-Code von a = 61h ≡ db 0x61
db 'Hallo'	die ASCII-Codes von H, a, l, l und o ≡ db 0x48, 0x61, 0x6c, 0x6c, 0x6f

Statt einzelnen Bytes können auch Words, DWords und QWords spezifiziert werden, Endianness muss dann zwingend beachtet werden:

```
dw 0x2135 ; = db 0x35, 0x21
dd 0x2135 ; = db 0x35, 0x21, 0x00, 0x00
dq 0x2135 ; = db 0x35, 0x21, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
```

3.1.1. Spezifikation von Zahlen

\$ Präfix auch möglich.

Base | **Suffix** | **Präfix**

Hex	H / X	0h / 0x
Dec	D / T	0d / 0t
Oct	Q / O	0q / 0o
Bin	B / Y	0b / 0y

3.2. BITS

NASM übersetzt fast jede Anweisung direkt in Binärzahlen und schreibt diese als Sequenz von Bytes in die Zielfeile

```
nop          ; 90h
not rax      ; 48h F7h D0h
mov rax, rbx ; 48h 89h D8h
```

Traditionell arbeitet der NASM im historischen 16-Bit-Modus, Operationen haben hier andere Opcodes als im 64-Bit-Modus. Der 64-Bit-Modus wird mit der Direktive **BITS 64** aktiviert (möglichst als 1. Zeile)

3.3. OFFSET

Nummeriert man die Bytes in der Ausgabe-Sequenz fortlaufend, erhält jedes Byte einen **Offset** (Adresse, Index).

```
...
dd 0x12345678
...
```

78 _h	56 _h	34 _h	12 _h
i	$i + 1$	$i + 2$	$i + 3$

\$ bezeichnet den aktuellen Offset, bevor die von der aktuellen Zeile generierten Bytes in die Ausgabe-Sequenz geschrieben werden.

```
db 'BSys 1?!'
dq $
; erzeugt          42 53 79 73 20 31 3f 21 08 00 00 00 00 00 00 00
```

3.4. LABELS

Am Anfang jeder Zeile in der Quelle kann ein Label stehen, das **nicht** in Maschinencode übersetzt wird:

```
my_text: db 'BSys 1?!'
current: dq current
; erzeugt auch          42 53 79 73 20 31 3f 21 08 00 00 00 00 00 00 00
```

Intern assoziiert der Assembler das Label w mit dem Offset $A(w)$ des nachfolgenden Befehls in der Ausgabedatei und ersetzt jede Verwendung von w in der Quelle durch $A(w)$ in der Ausgabe-Sequenz.

42 _h	53 _h	79 _h	73 _h	20 _h	31 _h	3f _h	21 _h	08 _h	00 _h	00 _h	00 _h	00 _h	00 _h	00 _h	00 _h
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
^ my_text								^ current							

NASM kann während der Übersetzung einfache Arithmetik ausführen:

```
length: dq after_my_text - my_text
my_text: db 'BSys 1?!'
after_my_text:
; erzeugt          08 00 00 00 00 00 00 00 42 53 79 73 20 31 3f 21
```

3.5. FLAT-FORM BINARIES

Per default erzeugt der NASM sogenannte *Flat-Form Binaries* (plain/pure/raw): Die reine Bytesequenz analog zum Quell-Text ohne Zusatzinformationen.

3.6. OBJEKT-DATEIEN

Objekt-Dateien enthalten darüber hinaus noch weitere Informationen, die später benötigt werden, u.a. die Symboltabelle.

Exportierte Symbole: mit `global` deklarierte Label

Importierte Symbole: mit `extern` deklarierte Label, haben keinen definierten Wert innerhalb der Objekt-Datei

Auf 64-Bit Linux sind Objekt-Dateien im Format ELF64 mit Endung `*.o`

```
nasm -f elf64 prog.asm -o prog.o
```

ELF64 kann mit dem Tool `objdump` analysiert werden:

```
objdump -t -d -Mintel prog.o
```

3.6.1. Symboltabelle

Die Symboltabelle enthält einen Eintrag pro Symbol.

```
global x, y
extern z
dq 0, 0
w: dq 0x12345678
x: dq 0xCAFEFACE
y: dq z
```

Offset	Attribute (g)lobal, (l)ocal	Sektion	Zusatzinformationen	Name
0000000000000010	l	.text	0000000000000000	w
0000000000000000		*UND*	0000000000000000	z
0000000000000018	g	.text	0000000000000000	x
0000000000000020	g	.text	0000000000000000	y

Das als extern deklarierte Label `z` erscheint als `*UND*` (undefined)

3.7. REGISTER

Es wird zwischen «General Purpose» und «Special Purpose» Register unterschieden.

General Purpose: Kann nach belieben verwendet werden

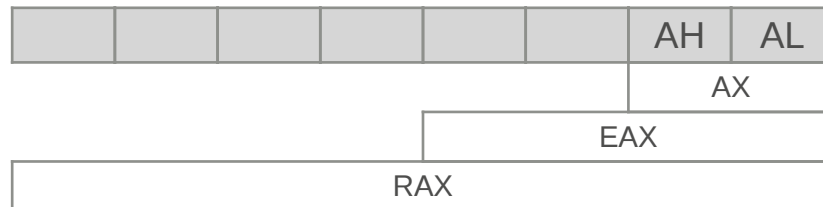
Special Purpose: Wird von der CPU für einen spezifischen Zweck verwendet, nur gewisse Operationen können darauf zugreifen.

Ursprünglich hatten Intel-Prozessoren (ab dem 8088) 16-Bit-Register, die in zwei 8-Bit-Register (AH AL) unterteilt wurden.

Man kann sowohl AX als auch AL oder AH in Instruktionen verwenden.

Mit der 32-Bit-Architektur wurden diese Register um namenlose 16-Bit erweitert (→ EAX), Mit der 64-Bit-Architektur um weitere 32-Bit (→ RAX).

Mit der ISA Erweiterung entstand aber auch eine neue Limitierung: ah, bh, etc. können nicht in allen Situation verwendet werden.



3.7.1. Allzweckregister

General Purpose Registers (GPRs)

RAX: Accumulator, für einige Rechenoperationen das einzige mögliche Register

RCX: Counter für Schleifen und Stringoperationen

RDX: Pointer für I/O-Operationen

RBX: Datenpointer

RSI, RDI: Quell- und Zielindizes für Stringoperationen

RSP: Stackpointer, Adresse des allozierten Stacks

RBP: Basepointer, Basis des Stackframe der Funktion

R8 - R15: Zusätzliche Register

Wichtig: Die Verwendungszwecke sind reine Konvention und nicht zwingend.

3.7.2. Spezialregister

Special Purpose Registers (SPRs)

RIP: Instruction Pointer, zeigt auf die nächste Instruktion

SS, CS, DS, ES, FS, GS: Segment Register (kaum noch verwendet)

RFLAGS: Flags die spezifische Situationen markieren (z.B. Integer Overflow)

CR0 - CR15: Control-Registers um die CPU zu konfigurieren

GDTR: Enthält die Adresse der GDT - verwendet für VM

LDTR: Enthält die Adresse der LDT - verwendet für VM

IDTR: Enthält die Adresse der IDT - verwendet für VM

TR: Enthält die Adresse des TSS - verwendet für Task Switching

3.7.3. Übersicht

63...56	55...48	47...40	39...32	31...24	23...16	15...8	7...0
RAX				EAX		AH	AL
						AX	
RCX				ECX		CH	CL

		CX	
RDY	EDX	DH	DL
		DX	
RBX	EBX	BH	BL
		BX	
RSP	ESP	-	SPL
		SP	
RBP	EBP	-	BPL
		BP	
RSI	ESI	-	SIL
		SI	
RDI	EDI	-	DIL
		DI	
R8	R8D	-	8B
		R8W	
R...	R...D	-	...B
		R...W	
R15	R15D	-	15B
		R15W	

Kompatibilität: 64-Bit ISA Instruktionen haben REX Präfix (Der NASM fügt diesen ein wenn nötig). Betroffen sind SPL, BPL, SIL und DIL.

Bsp: `mov ah, sil` entspricht `REX mov ah, sil`

Würde `mov spl, sil` bedeuten, wird nicht vom Assembler akzeptiert. Die Instruktionen könnten kodiert werden, hätten aber falsche Semantik.

3.8. INSTRUKTIONEN

Operationen benötigen unterschiedlich lange: Die schnellsten benötigen 1 Prozessorzyklus, die langsamsten mehrere 100.

Operationen, die auf den Speicher zugreifen müssen, müssen auf den Speicher warten: Ist der Operand im Cache: 4 bis 70 Zyklen, ansonsten: mehrere 100 Zyklen

Bestimmte Operationen könnten nur mit immensem Aufwand schneller gemacht werden (z.B. Division)

Instruktionen sind Binärzahlen, die die Operation und die Operanden codieren. Sie können auf Intel 64 unterschiedlich lang sein (1 bis 15 Byte). Die Anzahl und Grösse der Parameter hängen von der Operation ab. Die Länge einer Instruktion ist nicht in der Sequenz enthalten, eine Sequenz muss von Anfang an Instruktion für Instruktion durchgegangen werden, um diese richtig decodieren zu können.

3.8.1. Datentransfer-Operationen

`mov ziel, quelle`

Kopiert in das Ziel von der Quelle. Nach Ausführung der Operation enthalten Quelle und Ziel den gleichen Wert.

In ein **Register** kann man kopieren:

Woher	Befehl	Auswirkung
Von einem anderen Register	<code>mov rax, rbx</code>	Setze Inhalt von rax gleich Inhalt von rbx
Eine Konstante	<code>mov rax, 0x8000</code>	Setze Inhalt von rax gleich 8000h
Vom Speicher	<code>mov rax, [0x8000]</code>	Setze Inhalt von rax gleich Inhalt von 8000h ... 8007h

In den **Speicher** kann man kopieren:

Woher	Befehl	Auswirkung
Von einem Register	<code>mov [0x800], rbx</code>	Setze Inhalt von 8000h ... 8007h gleich Inhalt von rax
Eine Konstante	<code>mov qword, [0x8000], 5</code>	Setze Inhalt von 8000h ... 8007h gleich 5

aber **nicht** direkt vom Speicher in den Speicher.

Operandengrößen können explizit mit `byte`, `word`, `dword`, `qword`, etc angegeben werden.

Generell können die Operanden 8 Bit, 16 Bit, 32 Bit oder 64 Bit betragen, die Operanden müssen aber gleich gross sein, z.B.

`mov eax, ebx` OK, beide Register 32-Bit gross

`mov eax, rbx` Fehler, `eax` 32 Bit, `rbx` 64 Bit

Im Maschinencode gibt es für jede Operandengröße eine eigene Instruktion

Bit	Befehl	Code
8	<code>mov al, bl</code>	88 D8
16	<code>mov ax, bx</code>	66 89 D8
32	<code>mov eax, ebx</code>	89 D8
64	<code>mov rax, rbx</code>	48 89 D8

3.8.2. Adressierung

Speicherstellen können auf verschiedene Weisen spezifiziert werden.

Displacement: [`a`] Die Adresse `a` der Speicherstelle folgt unmittelbar

```
mov rax, [0x8000]
```

Base (Register): [`r`] Die Adresse der Speicherstelle steht in einem Register `r`

```
mov rbx, 0x8000
mov rax, [rbx]
```

Scaled Index: [`i * s`] Die Adresse `i * s` besteht aus einem Index `i` (Register) skaliert mit einer Konstante `s` (1,2,4 oder 8)

```
mov rcx, 0x1000
mov rax, [rcx * 8]
```

Jede Summe der drei vorherigen: Alle drei Adressierungsmodi können beliebig addiert werden

```
mov rbx, 0x4000
mov rcx, 0x1000
mov rax, [0x2000 + rbx + rcx * 2]
```

Wenn man nur die Adresse berechnen möchte, kann man die Operation `lea` (Load Effective Address) verwenden (greift nicht auf Speicher zu):

```
mov rbx, 0x4000
mov rcx, 0x1000
lea rax, [0x2000 + rbx + rcx * 2]
```

3.9. LINKER

Programme werden üblicherweise aus mehreren Assemblerdateien generiert. Der Assembler erzeugt aus einer Assemblerdatei eine Objekt-Datei. Der Linker `ld` erstellt aus einer oder mehreren Objekt-Dateien ein Executable.

Mit `ld -r prog.o prog2.o -o prog3.o` können wir partiell linken: Jedes Symbol erhält einen eigenen Platz im gelinkten Objekt; globale Symbole werden an einen neuen Offset verschoben. Durch `-r` wird kein Executable erzeugt, sondern eine weitere Objekt-Datei.

3.9.1. Einsprungspunkt

In jedem Executable muss der Einsprungspunkt (entry point) definiert werden (die Adresse, auf die der IP gesetzt wird, wenn das Programm gestartet wird). Die wird vom Linker gesetzt und ist standardmässig die Adresse des Labels `_start`, Kann auf der Kommandozeile geändert werden

```
ld -e main my_prog.o -o my_prog
```

3.9.2. Syscalls

Die Instruktion `syscall` übergibt die Ausführung an das OS. Wenn das OS fertig ist, geht es an der Stelle nach dem Syscall weiter. In `rax` übergibt man den Code für die OS-Funktion, für allfällige Parameter werden auf Intel 64 verwendet: `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. Codes und Parameter können je nach Architektur ändern; sind aber im allgemeinen recht konstant

3.9.3. Ende des Programms

Programme müssen explizit beendet werden: das OS weiss nicht, wann das Programm zuende ist. Dazu gibt es den OS-Syscall `exit`, üblicherweise Code 60 mit einem einzigem Parameter: einem 8-Bit Exit-Code. Ein Programm hat also immer folgenden Rahmen:

```
global _start
_start:
...
mov rax, 60      ; syscall exit
mov rdi, 0       ; exit code
syscall
```

4. C

4.1. TOOLCHAIN

Die Bestandteile der C Toolchain sind:

- 1) C Präprozessor
- 2) C Compiler
- 3) Assembler
- 4) Linker



Es lassen sich 3 Sprachebenen unterscheiden:

Präprozessor: definiert Direktiven, die im Programm vor dem eigentlichen Übersetzen als Textersetzung durchgeführt werden

Basiskonstrukte: bestimmen das Grundgerüst eines Programms, z.B. Variablen, Schleifen, Verzweigungen.

Standardbibliotheken: stellen Funktionen und Typen bereit, die die Basis-Funktionalität enthalten

4.2. PRÄPROZESSOR

Der Präprozessor verarbeitet die Input-Datei in mehreren Durchläufen. In jedem Durchlauf verarbeitet er die gesamte Datei einmal.

- 1) Entfernen aller Kommentare und umwandeln fortgesetzter Zeilen, die mit \ enden, in eine einzige Zeile.
- 2) Tokenization
- 3) Preprocessor directives + Macro expansion

4.2.1. Tokenization

Es gibt 5 Klassen von Tokens:

Bezeichner (identifiers): beginnt mit einem Buchstaben (a-zA-Z) oder _ gefolgt von einer Sequenz aus Buchstaben, _ oder Ziffern (0-9)

Präprozessor-Zahlen: Beginnt mit einer Ziffer Gefolgt von einer Sequenz aus Ziffern, Buchstaben, _, ., oder Exponenten (e+, E+, e-, E-, p+, P+, p-, P-). Vor der ersten Ziffer kann auch ein Punkt . stehen.

String- und Character-Literale: String-Literale beginnen und enden mit ", Character-Literale beginnen und enden mit '

Operatoren und Satzzeichen (punctuators): Jede der folgenden Zeichen bzw. Zeichenkombinationen gilt als Punctuator:

```

( ) [ ] { } . , : ; ? ... → # ##
= + - * / % & | ^ ~ ! << >> = ≠ < > ≤ ≥ && ||
*= /= %= += -= ++ -- <=> >=< &= |= ^=

```

Der Präprozessor ist greedy, d.h. er versucht immer das grösstmögliche Token zu bilden.

Sonstige: ?

4.2.2. Präprozessor-Direktiven

Ist das erste Token auf einer Zeile #, wird das nächste Token als Direktive interpretiert. Beide Token werden entfernt und die entsprechende Direktive ausgeführt. Die wichtigsten Direktiven sind:

include: Fügt einen Header ein

define: Definiert ein Makro

if: Bedingte Kompilation

else: Alternativer Zweig der bedingten Kompilation

endif: Ende der bedingten Kompilation

4.2.3. Includes

Präprozessor öffnet die Datei anhand des nächsten Tokens

- `#include <file.h>` sucht nur in den Systemverzeichnissen
- `#include "file.h"` sucht erst im aktuellen Verzeichnis und dann in den Systemverzeichnissen

Präprozessor führt Durchläufe 1 bis 3 für file .h durch und setzt Arbeit nach der Direktive in Originaldatei fort. Der Präprozessor kann dadurch mehrere Dateien zu einer **Translation Unit** zusammenführen

4.2.4. Macros

Es gibt objektartige und funktionsartige Makros. Objektartige Makros haben keine Parameterliste.

```
#define ANSWER 42
```

Der Präprozessor ersetzt im Programmtext nach der Definition des Makros jedes Token, das dem Makronamen entspricht, durch die Tokenliste. Nach der Ersetzung durchsucht der Präprozessor die Ersetzung auf weitere Makronamen und ersetzt diese. Taucht der eigene Makroname in der Ersetzung auf, wird er nicht ersetzt, um infinite Rekursion zu verhindern.

4.3. COMPILER

Der Compiler übersetzt eine C-Datei in eine Assembler-Datei.

Eine C Translation Unit ist eine Folge von Deklarationen und Definitionen von

- Globalen Variablen
- Funktionen
- Typen

Beispiel 2:

```
static int x = 0;      // [00] 8b 04 25 00 00 00 00    mov eax, [x]
extern void g();
void f() {
    if (x == 0) {      // [07] 85 c0                test eax, eax
                        // [09] 75 05                jne 0x10
        g();            // [0b] e8 00 00 00 00        call g
    }                  // [10] c3                ret
}
```

4.3.1. clang commands

Begriff	Definition
<code>clang prog.c -o prog</code>	Gesamte Toolchain
<code>clang prog.c</code>	Ohne Linker-Schritt, erzeugt prog.o
<code>clang -S prog.c</code>	Ohne Linker oder Assembler, erzeugt prog.s
<code>clang -masm=intel -S prog.c</code>	NASM-Kompatibles assembly
<code>clang -E prog.c prog.prep.c</code>	Nur Präprozessor

4.3.2. Deklaration und definition

Deklarationen haben keinen direkten Einfluss auf den erzeugten Byte-Stream (ähnlich wie Label in Assembler). Innerhalb einer Translation Unit darf jeder Bezeichner beliebig oft deklariert werden, solange die Deklaration gleich ist.

Eine Entität darf in einer Translation Unit nicht mehrfach definiert werden, auch nicht wenn die Definition exakt gleich ist.

4.3.3. Variablen

<i>global</i>	<i>extern</i>	<i>static</i>
Für jede globale Variable wird im Programm Speicher fix reserviert. Globale Variablen ohne Initialwert werden mit 0 initialisiert. Globale Variablen werden standardmässig exportiert: <pre>int c = 5;</pre> entspricht in Assembler <pre>global c c: dd 5</pre>	Globale Variablen, die aus anderen Objekt-Dateien verwendet werden sollen, werden mit extern deklariert: <pre>extern int c;</pre> entspricht in Assembler <pre>extern c</pre>	Variablen, die nicht exportiert werden sollen, werden mit static bezeichnet: <pre>static int c = 5;</pre> entspricht in Assembler: <pre>c: dd 5</pre>

<i>Bez. in Obj.-Datei</i>	<i>Deklaration in Asm</i>	<i>Bez. in C</i>	<i>Deklaration in C</i>
lokal	–	global, internal Linkage	static
global	global	global, external Linkage	–
–	extern	extern	extern

4.3.4. Quirks

```
// a.c
extern int x;
// b.c
short x;
// Wird kompiliert und gelinkt, kann aber crashen
// Im besten Fall passiert ein segfault
// Sollte in header Files definiert und überall inkludiert werden

static int x = 4;
extern int x;
// Compiler legt Speicher an für static int x, extern int x macht nichts
```